LayerTree: Constant-Size Exit and Eviction from a Shared Bitcoin UTXO

Olaoluwa Osuntokun

Lightning Labs
laolu@lightning.engineering

Abstract. Constructions that let many users share one Bitcoin output pay for the sharing at exit time: a user who leaves unilaterally publishes a chain of $O(\log N)$ transactions, the interior of the sharing tree must exist as pre-signed or template-committed transactions, and one unresponsive user forces the whole tree on-chain when it expires. We present LayerTree, in which a single output commits to an off-chain Merkle-sum tree of N balances and tapscript covenants enforce every state transition. Any member exits, and the operator evicts any expired member, in one transaction whose size is independent of N: 1,020 virtual bytes for a tree of 65,536 positions, about 2.5× less than unrolling an Ark- or CTV-style tree of the same size. Membership changes need no interactive signing round, the shared output never expires, and whole cohorts are added or swept in one constant-shape transaction (158 virtual bytes per evicted leaf).
The construction composes only opcodes already proposed for Bitcoin: the restored script opcodes of tapleaf version `0xC2`, transaction introspection, signature checks over stack messages and elliptic-curve tweaks. We give three complete realisations for decreasing opcode sets and show that `OP_CAT` plus one point-tweak opcode suffice, by having the script rebuild the BIP-341 signature message and check it with a generator-key signature. We show how Lightning channels are hosted inside leaves with prevout-independent commitments, so that a hub of N channels occupies one output. We prove conservation, proof soundness and unconditional payability in the idealised hash model and mechanise the proofs in Lean 4. A Python simulator, a Go implementation cross-checked byte for byte against it, and 338 executable checks accompany the design.

Keywords: Bitcoin · covenants · Merkle-sum trees · payment channels · layer two · formal verification

1 Introduction

Bitcoin's throughput is bounded by block space, and the settled answer is to move activity off-chain while keeping the chain as the court of last resort. Payment channels [20] do this for pairs; the open problem is doing it for *groups*: letting N users share one unspent transaction output (UTXO) without giving up the ability of each to leave alone. Every deployed or proposed approach to this problem

pays for the sharing when it is dissolved. In channel factories [2], Ark [10], timeout trees [11] and covenant trees built on `OP_CHECKTEMPLATEVERIFY` [21] or `OP_CHECKCONTRACTVERIFY` [9,8], the shared output is the root of a tree of *transactions*. A member who leaves unilaterally must publish the path from the root to its leaf, $O(\log N)$ transactions each paying a fee; the interior of the tree must exist in advance as pre-signed transactions (requiring every member to sign, and to be online) or as covenant templates; and when the tree expires, reclaiming the funds of a single unresponsive member means publishing everything.

This paper takes a different route. The tree stays entirely off-chain and is a tree of *data*, not transactions: a binary Merkle-sum tree whose leaves are balances and whose root is committed by one output. A member exits by presenting its leaf and its sibling path to the output itself. The spending script recomputes the root from the proof, checks it against the commitment, recomputes the root with the leaf removed, and requires the transaction to recreate the output under the new root with exactly the leaf's amount paid out. The member needs nobody's signature, the exit is one transaction, and its size grows with the depth of the tree rather than with the number of transactions in a path. The same covenant lets the operator evict a member whose lease has expired, graft a pre-built subtree of new members in one step, and sweep an expired cohort at a small fraction of the per-leaf cost.

What makes this possible is a set of script capabilities that have been proposed for Bitcoin but are not yet consensus: the restoration of the disabled string and arithmetic opcodes in a new tapleaf version ("Great Script Restoration", BIPs 440 and 441 [23,24]), transaction introspection (`OP_TX` [22]), signature verification over stack data (`OP_CHECKSIGFROMSTACK` [1]), and elliptic-curve point arithmetic [18,3]. We deliberately propose no opcode of our own. The design is an argument about what the candidate set buys, and part of the contribution is a precise account of how little of it is actually needed.

Contributions.

1. **Construction** (§4). A shared output committing to a Merkle-sum tree through a data-tweaked taproot internal key, with transitions for unilateral exit, eviction, batched exit, subtree grafting, cooperative update, single-use vouchers and cohort sweeps. Two tree profiles are specified: a dense operator-assigned tree and a key-addressed sparse tree with compressed proofs.
2. **Realisation in three opcode tiers** (§5). Every transition is implemented against the full candidate set, against a `0xC0`-only subset, and against the minimum we could find: `OP_CAT` and one point-tweak opcode. The last tier replaces introspection by a constructive trick, rebuilding the BIP-341 signature message in script and checking a signature under the generator point.
3. **Channel leaves** (§6). Lightning channels hosted in leaves, with commitment signatures that bind to no prevout (an `ANYPREVOUT` emulation from `OP_TX` and `OP_CHECKSIGFROMSTACK`) and an update/settlement state machine, so a channel's funding output need not exist until a dispute. We identify three tree-layer rules that hosting shared-key leaves requires.

4. **Security analysis with mechanised proofs** (§7). Conservation, proof soundness and payability are proved in the idealised hash model and checked in Lean 4. The proofs isolated two facts the informal argument had blurred: sums are conserved unconditionally, but exits are guaranteed only under grafted nodes that have preimages.
5. **Implementation and measurement** (§8). A tapscript simulator for the candidate opcodes, a Go implementation matched byte for byte against it, and measured costs: a unilateral exit of 1,020 vB at depth 16, a marginal batched exit of 665 vB, cohort eviction at 158 vB per leaf, and fee tables from 1 to 250 sat/vB.

The specification, code and proofs are public [19].

2 Background

Taproot and tapscript. A pay-to-taproot output [27] commits to an internal public key P and a Merkle root m of a *taptree* of scripts; the output key is $Q = P + H_{\mathsf{TapTweak}}(P \,\|\, m) \cdot G$. An output is spent either by a Schnorr signature [26] under Q or by revealing one script together with its Merkle path (the *control block*) and satisfying it. Tapscript [28] is a stack language without loops; every "loop" below is unrolled and its cost is linear in the number of iterations. A leaf's version byte selects its semantics.

Candidate opcodes. BIP-441 [24] defines tapleaf version `0xC2`, in which the string opcodes disabled in 2010 (`OP_CAT`, `OP_SUBSTR`, `OP_LEFT`, `OP_RIGHT`, bitwise operations and shifts) are restored and arithmetic is on unsigned numbers of arbitrary length, with BIP-440 [23] metering the runtime cost ("varops"). `OP_TX` [22] pushes selected fields of the spending transaction (its inputs' amounts and scripts, its outputs, lock time), optionally collated into one hash preimage. `OP_CHECKSIGFROMSTACK` [1] verifies a Schnorr signature over an arbitrary stack element. `OP_TWEAKADD` [3] computes the x coordinate of $P + t \cdot G$; the EC-ops draft [18] generalises this to point addition, scalar multiplication and negation. All are proposals; we treat them as given and note where the design depends on details that are still open.

Merkle-sum trees. A Merkle-sum tree [12,17] stores at every node the hash of its children together with the sum of the values below it, and hashes the children's sums into the parent. An inclusion proof for a leaf therefore proves not only membership but the leaf's contribution to the root sum, and the root sum is a commitment to the total. Sparse Merkle trees [4] address positions by hash, so that non-membership is an inclusion proof of an empty leaf.

3 Model and Goals

A *tree* has one *operator* and any number of *members*. Each member owns a *leaf*: a balance together with three public keys (an owner key that authorises the

member's actions, an exit key to which the balance is paid on exit, and an evictor key, normally the operator's, that may remove the leaf after a lease height). The operator holds no special key over the funds; it holds an *operator key* that authorises additions to the tree and co-signs cooperative changes. A *tree output* is the on-chain UTXO. Its amount equals the sum of all leaf balances, and its script commits to the tree root.

We assume the Bitcoin consensus and mempool rules with the opcodes of §2 active, collision-resistant tagged hashing (SHA-256), the discrete-logarithm assumption on secp256k1, and unforgeable Schnorr signatures. The adversary controls the operator, any subset of members, and the order of transactions within the rules; it may refuse to serve data. We want:

G1 Conservation. No transition creates value: the tree output's amount is always the sum of the leaf balances committed by its root, and every payout is funded by a leaf that the transition removes.

G2 Unilateral constant-size exit. A member holding its leaf and sibling path can always construct an accepted exit paying its balance, in one transaction whose size is independent of N, with no signature from any other party.

G3 Eviction. After a leaf's lease height its evictor can remove it, paying the balance to the leaf's exit key, so that an unresponsive member cannot pin the tree.

G4 Non-interactive membership. Adding or removing members requires no signing round among existing members and no pre-signed transactions.

G5 No expiry. The shared output has no lifetime; leases are per leaf and renewable.

Liveness of *joins* is not a goal: a malicious operator may refuse to add members, and members may refuse to be added. Data availability is a member's responsibility (§10).

4 Construction

4.1 The Tree

Fix a depth D. A tree has 2^D positions, each empty or holding a leaf. A leaf is a fixed 116-byte record:

$$\mathsf{le}_{64}(\mathsf{amount}) \,\|\, \mathsf{exit_key} \,\|\, \mathsf{owner_key} \,\|\, \mathsf{evictor_key} \,\|\, \mathsf{le}_{32}(\mathsf{evict_after}) \,\|\, \mathsf{le}_{32}(\mathsf{nonce}) \,\|\, \mathsf{le}_{32}(\mathsf{seq}),$$

with keys as 32-byte x-only points, $\mathsf{evict_after}$ a block height, nonce a per-leaf identifier and seq a counter advanced by vouchers (§4.3). A *node* is a pair (h, s) of a 32-byte hash and an unsigned sum. A position holding leaf ℓ has node $(H_{\mathsf{leaf}}(\ell), \ell.\mathsf{amount})$; an empty position has node $E_0 = (0^{32}, 0)$. The parent of (h_L, s_L) and (h_R, s_R) is

$$\big(H_{\mathsf{node}}(h_L \,\|\, \mathsf{enc}_{64}(s_L) \,\|\, h_R \,\|\, \mathsf{enc}_{64}(s_R)),\ s_L + s_R\big), \qquad \mathsf{enc}_{64}(s) = \mathsf{le}_{64}(s) \,\|\, \mathtt{0x01}.$$

H_{leaf} and H_{node} are BIP-340 tagged hashes with distinct tags. The encoding $\mathsf{enc}_{64}(s)$ is chosen so that a script can produce it from a computed sum with a single addition: under BIP-441 numerics the minimal encoding of $s + 2^{64}$ is exactly $\mathsf{le}_{64}(s) \,\|\, \mathtt{01}$ for $s < 2^{64}$ (Lemma 1). We write E_h for the root of the empty tree of height h; these constants are hard-coded in scripts.

The *sibling path* of a position is the list of D pairs $(\mathsf{dir}_k, (h_k, s_k))$, one per level, giving the direction taken and the sibling node not taken. $\mathsf{fold}(n, \pi)$ combines a node n with a path π level by level, hashing n with its sibling in the order the direction bit gives and adding the sums, and returns the implied root. The two facts that drive the design are that the honest path folds a leaf's node to the root, and that folding a *different* node up the *same* path yields the root of the tree with that position replaced (§7).

Profiles. In the *dense* profile positions are assigned by the operator and a path carries explicit direction bits; each level costs 88 bytes of script and 41 bytes of witness. In the *sparse* profile a leaf's position is the low D bits of $H_{\mathsf{pos}}(\mathsf{owner_key} \,\|\, \mathsf{nonce})$, direction bits are derived in script from the position hash, and empty siblings are elided from the witness (a one-byte marker in place of a node; the script substitutes E_k). The sparse profile is the Merkle-sum sparse Merkle tree of Taproot Assets [17] truncated to $D = 32$ or 64 levels: it gives key-addressed positions, a history-independent root and structural uniqueness of leaves, at about twice the per-exit cost of the dense profile (Table 2). A 256-level tree costs about twelve times as much and buys nothing beyond the ability to use a raw key as a position.

4.2 The Tree Output

Let r be the root hash and S the root sum. The tree output is a taproot output with internal key

$$P = \mathsf{NUMS} + H_{\mathsf{state}}(r) \cdot G,$$

where NUMS is the BIP-341 nothing-up-my-sleeve point, so the key path is unspendable and every spend goes through a script. Its taptree contains one leaf per supported transition script; the taptree's Merkle root m identifies the rule set and is constant for the tree's life. We write $\mathsf{TreeSPK}(r, m)$ for the resulting scriptPubKey. A tree output for state (r, S) is an output with script $\mathsf{TreeSPK}(r, m)$ and amount S.

Every transition script receives r and m in its witness, recomputes $\mathsf{TreeSPK}(r, m)$, and checks that it equals the scriptPubKey of the input being spent, read with `OP_TX`. This binds both the root the proofs are checked against and the script set the successor output will carry. The scripts cannot hard-code m because m depends on them; taking m from the witness and verifying it resolves the circularity, and binding it is security critical: a spender who could substitute m' would recreate the tree under scripts of its choosing.

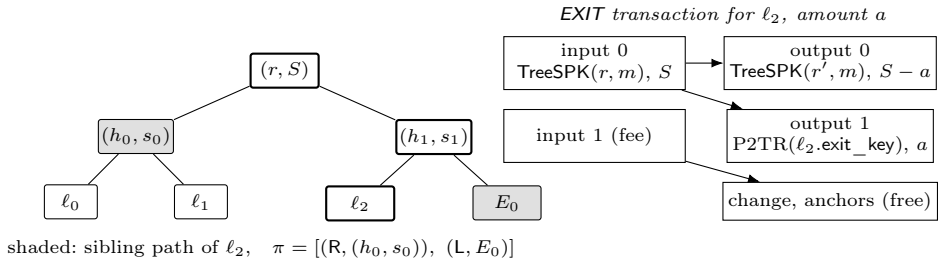


Fig. 1. A depth-2 tree and the exit of ℓ_2. The output commits to (r, S) through $\mathsf{TreeSPK}(r, m)$. The script checks $\mathsf{fold}((H_{\mathsf{leaf}}(\ell_2), a), \pi) = (r, S)$, computes $r' = \mathsf{fold}(E_0, \pi)$, verifies a signature by $\ell_2.\mathsf{owner_key}$, and constrains outputs 0 and 1; everything else in the transaction is free.

$\mathsf{Step}(\pi, n_{\text{old}}, n_{\text{new}}, (r, S))$

1 : $(r_0, S_0) \leftarrow \mathsf{fold}(n_{\text{old}}, \pi)$ // $D - h$ unrolled levels
2 : **assert** $r_0 = r \wedge S_0 = S$ // CHECK_OLD: S is input 0's amount
3 : $(r', S') \leftarrow \mathsf{fold}(n_{\text{new}}, \pi)$ // same siblings, same sums
4 : **return** (r', S')

$\mathsf{Final}((r', S'), m)$

1 : **if** $(r', S') = E_D$ **then return** // terminal: output 0 free
2 : $(a_0, \mathsf{spk}_0) \leftarrow$ OP_TX(output 0)
3 : **assert** $\mathsf{spk}_0 = \mathsf{TreeSPK}(r', m) \wedge a_0 = S'$

Fig. 2. The Merkle-sum step shared by all transitions, and the output rule. A transition is a sequence of steps, each proving its old node against the root the previous step produced (CHECK_PREV), followed by Final. Authorisation and payout checks are specific to each transition.

4.3 Transitions

Every transition spends the tree output as input 0 and recreates it as output 0 with the new root and the new total; payouts, if any, occupy outputs $1, \ldots, K$; all other inputs and outputs are unconstrained, so fees, change and anchors are free. The common core of every transition is the *step* of Figure 2: given a path π, an old node and a new node, check that the old node folds to the committed root and compute the root the new node folds to.

EXIT. Removes one leaf ℓ: the old node is $(H_{\mathsf{leaf}}(\ell), \ell.\mathsf{amount})$, the new node is E_0. Output 1 must pay to P2TR(ℓ.exit_key). The witness carries a flag selecting one of two authorisations. An *owner exit* requires a signature over the transaction by $\ell.\mathsf{owner_key}$ and permits output 1's amount to be at most $\ell.\mathsf{amount}$ (the difference is available as fee, and the owner consented to it by signing). An *eviction* requires a signature by $\ell.\mathsf{evictor_key}$, a lock time of at least $\ell.\mathsf{evict_after}$,

and output 1's amount *equal* to ℓ.amount: an evictor may remove but never shortchange. The signature must cover all outputs (`SIGHASH_DEFAULT` or `ALL`); otherwise a relayer could redirect the fee slack. An *exact-payout variant* $\mathsf{EXIT}^{=}$ requires equality for owner exits too; it is a tree-creation parameter and matters for shared-key leaves (§6).

JOIN$_h$. Grafts a subtree of height h into an empty aligned slot: the old node is the constant E_h, the new node $(h_{\text{new}}, s_{\text{new}})$ is supplied by the operator and authorised by its signature, and output 0's amount rises by s_{new}, which other inputs must fund. With $h = 0$ this adds one leaf; with $h = 8$ it adds up to 256 members for the same constant script cost. The covenant does not verify the interior of the grafted subtree, and cannot: prospective members verify their own paths inside it before funding the join (§7). JOINK_K sequences K single-leaf joins under one operator signature, which is the batching primitive of the sparse profile, where positions are not operator-assignable and grafting is unavailable.

UPDATE. Replaces one leaf by another at the same position with signatures from both the owner and the operator; the output amount moves by the difference. This is the splice, the lease renewal and the ownership transfer. A variant UPDATE^{x} additionally requires a signature by the *old* leaf's exit key, for leaves whose exit key is shared with a third party (§6).

CLAIM. Executes a *voucher*: an off-chain, owner-signed instruction to carve a new leaf out of an existing one. The voucher is a BIP-340 signature by the source leaf's owner over $H_{\mathsf{voucher}}(m \,\|\, \ell_{\text{src}} \,\|\, \ell_{\text{claim}})$, verified with `OP_CHECKSIGFROMSTACK`. The transition is two steps: the source leaf's amount is reduced by the claim amount and its seq advanced by one (or the position emptied if drained), then the claim leaf is placed at an empty position. The total is unchanged. Replay is impossible because the first step changes the source leaf, so the signed message never again matches a leaf in the tree; the seq field prevents a balance that later returns to an earlier value from re-enabling an old voucher. Vouchers are the mechanism for operator-to-member push payments without the operator's later participation; they are optional.

Composition. EXIT_K applies K exits in one transaction, each authorised independently, paying outputs $1..K$; the k-th step proves its leaf against the root produced by step $k-1$. $\mathsf{EVICT_SUBTREE}_h$ is a *shape-specialised multiproof*: it takes one sibling path to an aligned subtree of height h and the 2^h leaves below it (or empty markers), recomputes the subtree's root from them in post-order, checks each leaf's evictor key equals the operator key and its lease has expired, pays each leaf to its exit key at the next unused output, and replaces the subtree by E_h. It hashes the subtree's internal nodes once instead of once per leaf, which is what makes cohort eviction cheap (§8). A generic flag-driven multiproof was measured and rejected: it does not reduce script size, since script has no loops to share.

Table 1. The three opcode tiers. Sizes are bytes of script / witness data for EXIT_1 in the dense profile at $D = 16$.

	FULL	**MID**	**MIN**
Tapleaf version	`0xC2`	`0xC0`	`0xC0`
Numerics	unsigned, any length	4-byte `CScriptNum`	4-byte `CScriptNum`
Beyond BIP-342	`OP_CAT`, splice, bit ops; `OP_TX`; `OP_CSFS`; EC point ops	`OP_CAT`; `OP_TX`; `OP_CSFS`; `OP_TWEAKADD`	`OP_CAT`; `OP_TWEAKADD`
Introspection	`OP_TX`	`OP_TX`	constructive (§5.3)
Leaf on the stack	one 116-byte element	seven fields, re-joined	seven fields, re-joined
Sum limit per tree	none	2^{31} sat	2^{31} sat
Vouchers (CLAIM)	yes	yes	no
Channel leaves	yes	yes	no
Transaction shape	free	free	fixed (2 inputs)
EXIT_1, $D = 16$	2,101 / 902	3,299 / 978	3,572 / 1,175

Terminal transitions. If the new root is E_D the tree is empty and output 0 is unconstrained, so a tree can be dissolved without leaving a zero-value output.

5 Script Realisation and Opcode Tiers

Every transition of §4.3 is realised three times, against three opcode sets (Table 1), so that the design's dependence on each candidate opcode is measured rather than asserted. The tree format is identical in every tier; only the scripts differ.

5.1 The FULL Tier

With `0xC2` the leaf is a single 116-byte stack element whose fields are read with `OP_SUBSTR`; sums are added with arbitrary-length arithmetic and padded with $\langle 2^{64} \rangle$ `OP_ADD` (Lemma 1); each LEVEL of the fold is 88 bytes: concatenate the two 41-byte encodings in the order the direction bit gives, tagged-hash, add the sums. TreeSPK is recomputed with the EC opcodes: $H_{\mathsf{state}}(r)$ is multiplied onto G, added to NUMS, tweaked by the taptree hash, and its x coordinate wrapped in a P2TR script. The whole EXIT_1 prologue, including both introspections and the key recomputation, is under 700 bytes; depth costs 88 bytes per level.

5.2 The MID Tier

Tapleaf `0xC0` keeps BIP-342 semantics: no splice opcodes and 4-byte signed numbers. Two gadgets bridge the gap. Fields arrive as separate witness elements, each checked with `OP_SIZE` and re-joined with `OP_CAT` before hashing. A computed sum is widened to a fixed 8 bytes by a witness-supplied zero pad z and filler w with $z \,\|\, w =$ `00000000` and $|x| + |z| = 4$, so that $x \,\|\, z$ is the little-endian 4-byte form of x (PADZ). `MINIMALDATA` policy forbids using raw little-endian fields as numbers, so lock times and positions arrive as minimal numbers plus a pad. The cap of 2^{31} satoshis per tree (about 21.47 BTC) is the price of 4-byte arithmetic. Everything in the design survives, at about 1.6× the script size. The tier assumes `OP_TX` is available in `0xC0`, which the draft leaves open.

5.3 The MIN Tier: Introspection from a Signature

The covenant needs to read the spending transaction: the input's scriptPubKey and amount, and output 0 and output 1. Without `OP_TX` the only opcode that sees the transaction is `OP_CHECKSIG`, which verifies a signature over the BIP-341 signature hash. We turn that into introspection. The script *constructs* the signature message from values it has derived (the recomputed $\mathsf{TreeSPK}(r, m)$, the input amount, the new tree output and the payout) together with raw witness pieces for everything it does not constrain (other inputs, prevouts, sequence numbers, version, lock time, tapleaf hash), hashes it as BIP-341 prescribes, and then demands a Schnorr signature under the generator G with nonce G. For public key G and nonce $R = G$ the challenge is $e = H_{\mathsf{challenge}}(G_x \,\|\, G_x \,\|\, \mathsf{msg})$ and the verification equation $s \cdot G = R + e \cdot P = (1 + e) \cdot G$ has the unique solution $s = e + 1$. The script computes e from its constructed message, forms s, and calls `OP_CHECKSIG` with the signature $G_x \,\|\, s$ and public key G_x. Because `OP_CHECKSIG` hashes the *real* transaction, the constructed message must equal the real one, so every field the script fixed is enforced.

Without arbitrary-precision arithmetic, $e + 1$ is formed by grinding: the spender varies a free field (the fee input's sequence number) until the last byte of e is `0x00` and supplies the first 31 bytes; the script checks $e_{[0,31)} \,\|\, \mathtt{00} = e$ and uses $e_{[0,31)} \,\|\, \mathtt{01}$ as s. This costs 256 hashes on average and nothing on-chain. The transaction shape is fixed per script (two inputs; outputs: tree, payouts, then free extras), the tree output key is recomputed with two `OP_TWEAKADD`s, and EXIT_1 costs 3,572 bytes at $D = 16$, 43% more than FULL. Vouchers are impossible (there is no signature over stack data) and so are channel leaves.

The result is a precise minimality statement: BIP-342 tapscript plus `OP_CAT` plus one point-tweak opcode realise the covenant. Every restored splice and bit opcode, `OP_TX` itself and the EC point opcodes are conveniences, worth between 260 and 1,470 witness units per exit.

6 Channel Leaves

A leaf whose exit key is a 2-of-2 between its owner and a counterparty (normally the operator) can host a payment channel that exists only off-chain while the leaf is in the tree and whose funding output materialises at the payout of the transaction that removes the leaf. A tree of N such leaves is a Lightning hub of N channels behind one UTXO, with opening, resizing and closing of channels costing no on-chain transaction except at dispute.

Rebindable commitments. The funding outpoint is unknown when channel states are signed, and it changes with every tree transition. Commitment transactions must therefore be authorised by signatures that do not commit to the prevout, which `OP_TX` and `OP_CHECKSIGFROMSTACK` provide as an emulation of `SIGHASH_ANYPREVOUT` [6]: the script collates, with one `OP_TX` call, the fields the signature is meant to cover (version, lock time, the current input's sequence, the number of inputs, and all outputs), prefixes a per-channel context constant baked

REBIND_n (tapleaf 0xC2, 81 B)	SETTLE_n (tapleaf 0xC2, 80 B)
$\langle S_0 + n + 1 \rangle$ `OP_CHECKLOCKTIMEVERIFY OP_DROP`	$\langle \mathsf{csv_delay} \rangle$ `OP_CHECKSEQUENCEVERIFY OP_DROP`
$\langle \mathsf{SEL_CHAN_UPD} \rangle$ `OP_TX` // collated blob	$\langle \mathsf{SEL_CHAN_SET} \rangle$ `OP_TX` // + input amount, spk
$\langle \mathsf{ctx_{upd}} \rangle$ `OP_SWAP OP_CAT OP_SHA256`	$\langle \mathsf{ctx_{set}} \rangle$ `OP_SWAP OP_CAT OP_SHA256`
$\langle P_{\mathsf{chan}} \rangle$ `OP_CHECKSIGFROMSTACK`	$\langle P_{\mathsf{chan}} \rangle$ `OP_CHECKSIGFROMSTACK`

Fig. 3. The two leaves of a channel leaf's update output for state n. A later update binds through REBIND_n; the settlement of state n binds through SETTLE_n after the dispute delay. P_{chan} is a MuSig2 [16,15] key of the two parties.

into the script, hashes, and verifies a BIP-340 signature under the channel key with `OP_CHECKSIGFROMSTACK`. The context commits to the tree identifier m, the leaf identity, the channel key, the capacity and a splice epoch, so a state signed for one channel or capacity never verifies for another. Committing the number of inputs (one) prevents two funding outputs with the same script from being spent by one commitment.

State machine. Penalty channels cannot be used: a penalty transaction must be pre-signed against a specific commitment outpoint, which is exactly what is unknown. Channel leaves use the update/settlement ratchet of eltoo [5,14] (Figure 3). The update transaction for state n has lock time $S_0 + n$ in the timestamp range and spends either the funding output or the update output of any earlier state $j < n$, whose REBIND_j leaf admits only lock times $\geq S_0 + j + 1$. The settlement transaction spends state n's update output after a relative delay through SETTLE_n, whose signature also commits to the spent script (`ANYPREVOUT` rather than `ANYPREVOUTANYSCRIPT`), so it binds only to its own state. Both transactions are zero-fee version-3 transactions with an ephemeral anchor [29]; fees are paid by a child. Because the taptree of an update output is a pure function of the context and n, and n is published in the lock time, no party needs to retain script data to react to a published state.

HTLCs and routing. Settlement outputs include one output per in-flight HTLC with the usual hash and timeout branches; no second-stage transactions are needed because the settlement is itself delayed. Payments between two channel leaves of the same tree are ordinary HTLC forwards through the operator and never touch the tree output. The deadline budget an HTLC needs is the time to materialise the funding output (one tree transaction), one block for the update transaction, the dispute delay and the sweep: roughly 220 blocks with a 144-block delay, against the Lightning default of 40 to 80.

Three tree-layer rules. Working the construction through exposed places where the tree rules were too lenient for a leaf whose balance is not the owner's alone. (i) The owner's fee-from-payout option would let the owner underfund the channel and hold the counterparty's balance hostage, so channel leaves require exact payouts and trees hosting them should use $\mathsf{EXIT}^{=}$, which is visible in m. (ii) A

voucher lets the owner carve value out of a leaf with a lone signature, so channel leaves set seq to the one value whose successor does not fit in four bytes and thereby cannot be claimed: `0xFFFFFFFF` under BIP-441 numerics but `0x7FFFFFFF` under `CScriptNum`, since `ff ff ff ff` there encodes $-(2^{31}-1)$, whose successor fits (§7). (iii) The lease height is the height before which the counterparty can enforce nothing on-chain, so a routing channel leaf must set it to zero. For channels between two members, UPDATE co-signed by owner and operator could splice the leaf down behind the counterparty's back; UPDATE^x closes this by requiring the old exit key's signature as well.

Channel leaves need `OP_TX` and `OP_CHECKSIGFROMSTACK` and therefore the FULL or MID tier. The MIN tier's constructive introspection necessarily commits to the prevout and cannot be rebound.

7 Security Analysis

We work in the idealised hash model: H_{leaf} and H_{node} are injective on the domains the protocol uses, and digests under different tags never coincide or equal the zero string. This is the standard treatment of collision resistance for Merkle constructions; a proof under injectivity holds against any adversary that cannot exhibit a collision, which is the event the assumption excludes. The theorems below are stated for the abstract transition rules of §4.3; all are mechanised in Lean 4 [13] in about 1,300 lines without Mathlib, and the names in brackets are the Lean identifiers. The correspondence between the rules and the tapscript bytecode is established by execution (§8), not proof.

Lemma 1 (Encodings). *For $s < 2^{64}$, the minimal BIP-441 encoding of $s + 2^{64}$ is $\mathsf{le}_{64}(s) \,\|\, \mathtt{01}$; leaf and node serialisations are injective under the field bounds; the minimal encoding of 2^{32} has five bytes while that of $2^{32}-1$ has four; and the `CScriptNum` of $-(2^{31}-1)$ is `ff ff ff ff` with successor `fe ff ff ff`. [enc64_eq_minimal,serializeLeaf_inj,serializeNode_inj,minimalLE_two_pow_32_length, scriptNum_neg_max31_succ]*

Theorem 1 (Binding). *Roots bind trees: two trees of the same depth with equal roots are equal. Any node n and path π with $\mathsf{fold}(n, \pi)$ = the root of a tree T are the root of one of T's subtrees and that subtree's honest path. In particular the sum in n is the true sum of that subtree, and a leaf node that verifies with a full-length path is the leaf at that position. [root_inj,fold_sound]*

Proof (sketch). Induction on the path. A non-empty path claims an internal node, so the tree is not a tip by tag separation. Injectivity of H_{node} on the parent's preimage equates both children's hashes *and* sums with the claimed ones, and the induction hypothesis applies to the child the direction bit selects. This is the only place the hash axioms are used.

Theorem 2 (Conservation). *For any sequence of steps from root (r, S) to (r', S'), $S' + \sum_{\text{steps}} s_{\text{old}} = S + \sum_{\text{steps}} s_{\text{new}}$. Hence for EXIT_K, $S' + \sum_k \text{payout}_k \leq$*

S, with equality under exact payouts; for JOIN$_h$*, $S' = S + s_{\text{new}}$; for* UPDATE*, $S' + a_{\text{old}} = S + a_{\text{new}}$; and* CLAIM *leaves S unchanged. [Steps.sum,ExitK.no_inflation, ExitK.exact_conservation,join_sound,update_sound,claim_sound]*

Proof (sketch). A fold adds exactly the sibling sums it is shown, so the two folds of a step differ by $s_{\text{new}} - s_{\text{old}}$; the rest is induction on the sequence. No hash assumption is needed: the covenant conserves sums whatever nodes it is shown.

Theorem 3 (Payability). *For any tree T and position p holding leaf ℓ, the honest witness (the leaf, its path, E_0 as new node) is a valid step from T's root to the root of T with p emptied; the new sum is the old sum minus ℓ.*amount*; and every other position is unchanged. Together with Theorem 2: whoever holds a leaf and its path can always construct an accepted* EXIT *paying the leaf amount, without anyone's cooperation and without touching anyone else's leaf. [payable]*

Theorem 4 (Grounded invariant). *If the current root is the root of a tree and every step's new node is the root of some tree of the step's height, the final root is the root of a tree of the same depth. Leaf-level steps satisfy the hypothesis by construction. [Steps.sound]*

Theorem 4 is where the mechanisation sharpened the argument. A JOIN$_h$ graft is grounded only if the operator's $(h_{\text{new}}, s_{\text{new}})$ has a preimage tree. If it does not, Theorem 2 still holds, so the on-chain amount still equals the committed sum, but nothing below the node can ever be proven: the grafted value is frozen. The covenant protects the *sum*; the members protect their *membership* by verifying their own paths inside the subtree before funding the join, which the specification makes mandatory. A second sharpening: a CLAIM requires the claim position to be empty *after* the source step, not before, so a source drained to zero may serve as the claim position in the same transaction; the model shows this is harmless.

Root binding to the output. The scripts conclude that the witness root and taptree are the committed ones from $\mathsf{TreeSPK}(r', m') = \mathsf{spk}_{\text{in}} = \mathsf{TreeSPK}(r, m)$, which needs TreeSPK injective in (r, m); this follows from the discrete logarithm assumption and collision resistance of H_{state} and the taproot tweak hash. Without binding m a spender could recreate the tree under scripts of its own [witness_root_binds].

Authorisation and lock times. These decide who may perform a transition, not what it does to the tree, and the theorems carry them as opaque propositions. The requirement that exit signatures cover all outputs matters because the covenant only bounds output 1's amount for owner exits; with `SIGHASH_NONE` a relayer could divert the slack.

Concurrency. All transitions spend one UTXO, so at most one confirms per tree per block position and conflicts resolve by fee; an owner's exit conflicts with a pending operator join and either can replace the other under BIP-125 rules. Default mempool policy (25 descendants) bounds uncooperative exits to about 25 per confirmed tree output, i.e. per block. This is the design's honest weak

Table 2. Measured cost of EXIT_1 (FULL tier) by profile and depth. Witness units (WU) include the 129-byte control block; the transaction adds a fee input, a change output and overhead (≈ 950 WU). The Ark/CTV column models a unilateral exit from a radix-2 transaction tree as D transactions of 150 vB plus a 111 vB leaf claim.

profile	D	script (B)	witness (B)	total (WU)	tx (vB)	Ark/CTV (vB)
dense	8	1,397	574	2,100	762	1,311
dense	12	1,749	738	2,616	891	1,911
dense	16	2,101	902	3,132	1,020	2,511
dense	20	2,453	1,066	3,648	1,149	3,111
sparse	32	5,147	326	5,602	1,637	4,911
sparse	64	9,499	326	9,954	2,725	—
MID dense	16	3,299	978	4,406	1,338	
MIN dense	16	3,572	1,175	4,876	1,456	

spot: a tree of 10,000 channel leaves needs about 400 blocks to force-close all of them, while HTLC deadlines run. Operators should size trees by exit throughput rather than depth and shard a deployment across trees.

Data availability. Members must retain their leaf and path and process every spend of the tree output; the witness of any transition contains the full sibling path it used, so a member can update its own path from the chain alone and never depends on the operator for the data needed to exit. Members who fall behind can catch up from the chain.

8 Evaluation

Implementation. The reference implementation is a Python package with a tapscript interpreter for both leaf versions that models BIP-441 numerics, `OP_TX`, `OP_CHECKSIGFROMSTACK`, the EC opcodes and BIP-440 varops accounting, computes the real BIP-341 signature message, and enforces `MINIMALDATA` policy. Every script of every tier is executed against a reference tree in both profiles with positive and negative cases targeting each rule (forged leaf, stale root, attacker-chosen m, unconserved amount, underfunded join, occupied position, early eviction, evictor shortfall, unsigned update, voucher replay, over-claim, wrong signer, stale second path in a batch, and so on): 272 FULL, 59 MID and 7 MIN checks. The suite fails if any script uses an opcode outside the candidate set. A Go implementation on `btcd` and `lnd` generates the same scripts and is compared byte for byte against 30 golden vectors; its BIP-341 signature hash equals the simulator's, which is what makes the MIN tier's constructive introspection testable end to end. The Lean development of §7 builds with the core library alone and prints an axiom audit.

Exit cost. Table 2 and Figure 4 give the measured sizes. In the dense profile a unilateral exit costs 762 vB at $D = 8$ and 1,149 vB at $D = 20$: a 4,096-fold increase in tree size raises the exit by 51%, about 32 vB per level, because only the script and one sibling per level grow. The Ark/CTV model grows by a full

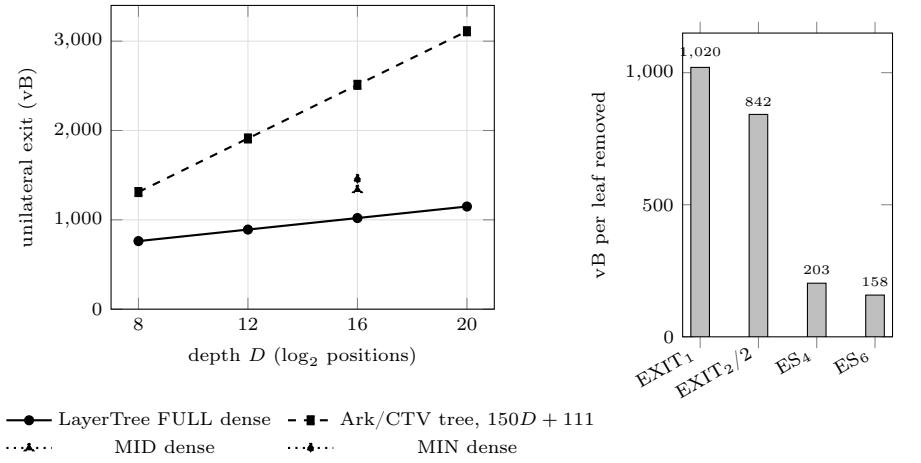


Fig. 4. Left: unilateral exit cost against tree depth, measured for LayerTree and modelled for a transaction tree. Right: per-leaf cost of removal at $D = 16$: a single exit, the average of a two-exit batch, and cohort sweeps $\mathsf{EVICT_SUBTREE}_4$ (13 leaves) and $_6$ (51 leaves).

transaction per level. LayerTree breaks even near $D = 5$ and is 2.5× cheaper at $D = 16$. Tier costs more than depth: MID is 31% and MIN 43% over FULL at the same depth. The sparse profile at $D = 32$ costs 1.6× the dense profile at $D = 16$, and witness data is nearly constant in D because empty siblings are elided.

Batching and cohorts. A second exit in the same transaction costs 665 vB marginally against 1,020 vB alone, since the prologue, key recomputation and skeleton are shared. Grafting is the decisive difference between the profiles for onboarding: a dense JOIN_h adds any number of members for a constant $\approx$ 2,000 WU, while sparse joins cost $\approx$ 5,000 WU per leaf however they are batched. Cohort sweeps do for removal what grafting does for addition: $\mathsf{EVICT_SUBTREE}_4$ removes 13 leaves at 203 vB each and $\mathsf{EVICT_SUBTREE}_6$ removes 51 at 158 vB each, a 5 to 6.5× reduction over individual evictions that improves with cohort size because the subtree's interior is hashed once. Operators therefore graft members who share a lease expiry into one aligned subtree, and both add and remove the cohort in one constant-shape transaction each.

Fees. At 10 sat/vB a $D = 16$ exit costs 10,200 sat, so a balance of 1,020,000 sat exits for a 1% fee and 102,000 sat for 10%; at 50 sat/vB the figures are five times higher. Cohort sweeps keep small leaves evictable at high rates: 158 vB per leaf is 7,900 sat at 50 sat/vB. Dust is never the binding constraint for $D \geq 8$; the fee is. Varops usage is 6 to 8% of the BIP-440 budget in the dense profile and 8 to 25% in the sparse profile, dominated by the EC operations and the signature at small depths and by hashing at large ones.

Table 3. Comparison with shared-output constructions.

	LayerTree	Ark	Timeout tree	CTV/CCV tree
Unilateral exit	1 tx, $O(D)$ B	$O(\log N)$ txs	$O(\log N)$ txs	$O(\log N)$ txs
Evict one leaf	1 tx after lease	tree expires	tree expires	no
Interactive signing	none	round members	all	none
Shared output expiry	none	per round	per tree	per design
Batch add	1 tx, any subtree	per round	per tree	per tree
Off-chain payments	channel leaves, vouchers	out-of-round	channels	varies
Opcodes	GSR, `OP_TX`, CSFS, EC	none / CTV	CTV	CTV / CCV

Channel leaves. A force-close of a channel leaf at $D = 16$ is the exit (1,020 vB), an update transaction (153 vB) and its anchor child, and after the delay a settlement (204 vB plus 43 vB per HTLC) and its child: about 1,680 vB in total, of which the channel machinery is about 660 vB. It is roughly 13× a Lightning force close, and the difference is exactly the price of never having had the funding output on-chain; cooperative closes, splices and lease renewals cost one tree transition or nothing.

9 Related Work

Channel factories [2] share one output among N channels but require all N to sign every factory-level change and publish a transaction tree to exit. Ark [10] batches users into rounds whose virtual outputs are leaves of a pre-signed or CTV-committed tree; exit unrolls the branch, the round expires, and users must refresh. Timeout trees [11] and MATT-style covenant trees [8,9] replace signatures by templates but keep the transaction tree and its $O(\log N)$ exit. Statechains [25] transfer a whole UTXO off-chain but do not divide it. LayerTree differs from all of these in that the tree is data rather than transactions: exit is $O(1)$ transactions and $O(D)$ bytes, the interior needs no pre-signing, the operator can evict, and the output never expires. Table 3 summarises.

The Merkle-sum sparse Merkle tree is taken from Taproot Assets [17], where it commits to asset balances; here it is truncated in depth and its verification moved into script. The data-tweaked internal key follows the pattern of `OP_CHECKCONTRACTVERIFY` [9], realised from EC opcodes. The channel state machine is eltoo [5]; its `ANYPREVOUT` requirement [6] is met here by collated introspection and `OP_CHECKSIGFROMSTACK` rather than a new sighash flag, and unlike CTV-based LN-Symmetry no script data need be retained to respond to a published state. The constructive introspection of §5.3 is in the tradition of covenant tricks that recover transaction data from a signature check; we are not aware of a prior use of a generator-key signature with $s = e + 1$ to make the script's reconstruction binding.

10 Discussion and Limitations

What the covenant does and does not guarantee. It guarantees conservation and the payability of every leaf under a grounded node. It does not guarantee that the

operator will add you (a member who is never added has lost nothing), nor that the operator's grafted nodes have preimages (a member who funds a join without verifying its path inside the subtree has trusted the operator). The specification makes the check mandatory, and the Lean development shows it is load-bearing.

Exit serialisation. Every transition spends the same UTXO. Cooperative batching makes this a non-issue for a responsive operator; an uncooperative mass exit is bounded by mempool descendant policy to about 25 leaves per block. Trees should be sized by this throughput, not by depth, and channel-hosting trees by it against HTLC deadlines. This is a property of the single-UTXO design and cannot be engineered away within it; sharding is the answer.

Operator power. The operator cannot steal, freeze provable leaves, or block exits. It can evict any leaf after its lease and, in a cohort tree, many at once for about 160 vB each, which makes griefing cheap for it. Members who want a hard guarantee buy it with a future lease height and, for channel leaves, give up routing until it passes.

Opcode dependence. The design is hostage to proposals. Its strongest dependency is on a tapleaf version with restored string opcodes or, failing that, `OP_CAT`; `OP_TX` in `0xC0` is assumed for the MID tier and its selector values are drafts; the EC opcodes' operand order and cost are drafts. Every selector and opcode number is a named constant in the implementation.

Future work. The off-chain protocol (joins, cohorts, channel leaves under an adversarial scheduler) is a different kind of object from the covenant and wants a model checker; we have a design for a P model [7]. A verified interpreter for the tapscript subset would close the gap between the rules the Lean model proves and the bytecode the tests execute. Hash-locked vouchers give atomic swaps between the credit instrument and channel HTLCs inside one tree. Aggregate node keys would allow Ark-style cooperative subtree replacement in one signature, at the cost of the interactive round the design set out to avoid.

11 Conclusion

A shared Bitcoin output whose tree lives off-chain as data rather than transactions gives every member a one-transaction exit of about a kilobyte, gives the operator eviction and cohort sweeps, needs no pre-signing and never expires. The construction uses only opcodes already on the table, and the minimum it truly needs is `OP_CAT` and a point tweak. Its guarantees are theorems in the idealised hash model, checked mechanically, and its costs are measured by execution. The price is that all activity funnels through one UTXO, which bounds how fast a tree can be dissolved against its will; we regard that as the honest trade-off of the approach and the parameter operators must design around.

References

1. Black, B., Rubin, J.: BIP-348: CHECKSIGFROMSTACK. https://github.com/bitcoin/bips/blob/master/bip-0348.md (2024)
2. Burchert, C., Decker, C., Wattenhofer, R.: Scalable funding of bitcoin micropayment channel networks. In: Royal Society Open Science. vol. 5 (2018)
3. Code, R., et al.: BIP-449: OP_TWEAKADD. https://github.com/bitcoin/bips/ (2025)
4. Dahlberg, R., Pulls, T., Peeters, R.: Efficient sparse Merkle trees: Caching strategies and secure (non-)membership proofs. In: NordSec 2016. LNCS, vol. 10014, pp. 199–215. Springer (2016)
5. Decker, C., Russell, R., Osuntokun, O.: eltoo: A simple layer2 protocol for bitcoin. https://blockstream.com/eltoo.pdf (2018)
6. Decker, C., Towns, A.: BIP-118: SIGHASH_ANYPREVOUT for taproot scripts. https://github.com/bitcoin/bips/blob/master/bip-0118.mediawiki (2017)
7. Desai, A., Gupta, V., Jackson, E., Qadeer, S., Rajamani, S., Zufferey, D.: P: Safe asynchronous event-driven programming. In: PLDI 2013. pp. 321–332. ACM (2013)
8. Ingala, S.: Merkleize all the things. bitcoin-dev mailing list, https://merkle.fun (2022)
9. Ingala, S.: BIP-443: OP_CHECKCONTRACTVERIFY. https://github.com/bitcoin/bips/blob/master/bip-0443.md (2025)
10. Keceli, B.: Ark: An off-chain payment protocol. https://arkdev.info/ (2023)
11. Law, J.: Scaling lightning with simple covenants. bitcoin-dev mailing list, https://github.com/JohnLaw2/ln-scaling-covenants (2023)
12. Merkle, R.C.: A digital signature based on a conventional encryption function. In: CRYPTO 1987. LNCS, vol. 293, pp. 369–378. Springer (1988)
13. de Moura, L., Ullrich, S.: The Lean 4 theorem prover and programming language. In: CADE 2021. LNCS, vol. 12699, pp. 625–635. Springer (2021)
14. Myers, R., et al.: LN-Symmetry (eltoo) implementation notes. https://github.com/instagibbs/bolts/blob/eltoo_draft/XX-eltoo-transactions.md (2023)
15. Nick, J., Ruffing, T., Jin, E.: BIP-327: MuSig2 for BIP340-compatible multi-signatures. https://github.com/bitcoin/bips/blob/master/bip-0327.mediawiki (2022)
16. Nick, J., Ruffing, T., Seurin, Y.: MuSig2: Simple two-round Schnorr multi-signatures. In: CRYPTO 2021. LNCS, vol. 12825, pp. 189–221. Springer (2021)
17. Osuntokun, O.: Taproot assets: Merkle sum sparse merkle trees. https://github.com/lightninglabs/taproot-assets/blob/main/docs/bip-tap-ms-smt.mediawiki (2022)
18. Osuntokun, O.: Elliptic curve operations for bitcoin script. Draft BIP, https://github.com/bitcoin/bips/pull/1945 (2025)
19. Osuntokun, O.: LayerTree: Specification, reference implementation and formal model. https://github.com/roasbeef/layertree (2026)
20. Poon, J., Dryja, T.: The bitcoin lightning network: Scalable off-chain instant payments. https://lightning.network/lightning-network-paper.pdf (2016)
21. Rubin, J.: BIP-119: CHECKTEMPLATEVERIFY. https://github.com/bitcoin/bips/blob/master/bip-0119.mediawiki (2020)
22. Russell, R.: OP_TX: Transaction introspection for tapscript. Draft BIP, https://github.com/rustyrussell/bips/pull/1 (2025)
23. Russell, R., Towns, A.: BIP-440: Varops budget for script runtime constraint. https://github.com/bitcoin/bips/blob/master/bip-0440.md (2025)

24. Russell, R., Towns, A.: BIP-441: Restoration of disabled script opcodes (tapleaf 0xC2). https://github.com/bitcoin/bips/blob/master/bip-0441.md (2025)
25. Somsen, R.: Statechains: Off-chain transfer of UTXO ownership. In: Scaling Bitcoin 2018 (2018)
26. Wuille, P., Nick, J., Ruffing, T.: BIP-340: Schnorr signatures for secp256k1. https://github.com/bitcoin/bips/blob/master/bip-0340.mediawiki (2020)
27. Wuille, P., Nick, J., Towns, A.: BIP-341: Taproot: SegWit version 1 spending rules. https://github.com/bitcoin/bips/blob/master/bip-0341.mediawiki (2020)
28. Wuille, P., Nick, J., Towns, A.: BIP-342: Validation of taproot scripts. https://github.com/bitcoin/bips/blob/master/bip-0342.mediawiki (2020)
29. Zhao, G.: BIP-431: Topology restrictions for pinning. https://github.com/bitcoin/bips/blob/master/bip-0431.mediawiki (2024)

A The Lean Formalisation

This appendix describes the mechanised model behind §7 in enough detail that a reader can judge what has been proved, under what assumptions, and what has not. The development is about 1,300 lines of Lean 4 [13] in six modules and depends on nothing but the core library: no Mathlib, no external tactics. It builds in seconds and prints an axiom audit for every headline theorem; each depends only on propext, Quot.sound and, in one case, Classical.choice, and none on sorryAx. The code is in the artefact [19].

A.1 Design of the Model

The model states the *rules* of §4.3 as relations over the objects the scripts manipulate: nodes, paths, amounts. It does not model tapscript bytecode, signatures, keys or lock times. Authorisation is carried as an opaque proposition, and the input and output amounts the scripts read with OP_TX are parameters. Three choices keep the proofs short.

Depth as a type index. A tree of depth d is Tree d, a perfect binary tree whose tips are Option Leaf. Positions are Pos d h, paths from depth d down to a subtree of height h, so that Pos d 0 addresses a leaf and Pos d h addresses the slot a JOIN_h fills. Indexing by both endpoints avoids all arithmetic on depths: the recursions on getSub, setSub and pathSub pattern-match on the position and the tree simultaneously, and the one impossible case (a cons step below height 0) is dismissed by Pos.height_le.

```
inductive Tree : Nat → Type
  | tip : Option Leaf → Tree 0
  | bin {d : Nat} : Tree d → Tree d → Tree (d + 1)

inductive Pos : Nat → Nat → Type
  | nil {h : Nat} : Pos h h
  | cons {d h : Nat} : Bool → Pos d h → Pos (d + 1) h

def parent (a b : Node H) : Node H :=
  ⟨M.hNode a.hash a.sum b.hash b.sum, a.sum + b.sum⟩
```

```
def combine (n : Node H) (x : Bool × Node H) : Node H :=
  if x.1 then parent x.2 n else parent n x.2

def fold (n : Node H) : List (Bool × Node H) → Node H
  | [] => n
  | x :: rest => combine (fold n rest) x
```

Root-first paths. The model orders a sibling path from the root down, so fold is a structural recursion on the list and the soundness proof is a plain induction. The witness of §5 supplies the same pairs leaf-first, because the script consumes the deepest level first; nothing in the theorems depends on the order.

Hashes as parameters. Everything is parametric in a class HashModel whose fields are the two hash functions, the zero digest, and the assumptions the argument uses:

```
class HashModel (Leaf : Type) (H : outParam Type) where
  hLeaf : Leaf → H
  hNode : H → Nat → H → Nat → H
  empty0 : H
  hLeaf_inj : ∀ {l₁ l₂}, hLeaf l₁ = hLeaf l₂ → l₁ = l₂
  hNode_inj : ∀ {a₁ b₁ a₂ b₂ sa₁ sb₁ sa₂ sb₂},
    hNode a₁ sa₁ b₁ sb₁ = hNode a₂ sa₂ b₂ sb₂ →
      a₁ = a₂ ∧ sa₁ = sa₂ ∧ b₁ = b₂ ∧ sb₁ = sb₂
  hNode_ne_hLeaf : ∀ a sa b sb l, hNode a sa b sb ≠ hLeaf l
  hNode_ne_empty : ∀ a sa b sb, hNode a sa b sb ≠ empty0
  hLeaf_ne_empty : ∀ l, hLeaf l ≠ empty0
```

Injectivity is collision resistance idealised; the three inequalities are the tag separation that distinct BIP-340 tags provide. The encodings module discharges the serialisation half of each injectivity field (Lemma 1): given SHA-256 collision resistance, serializeLeaf_inj and serializeNode_inj are what remains. Nothing quantifies the adversary's advantage; a proof under injectivity transfers to any adversary that cannot exhibit a collision.

A.2 Theorem Statements

The statements below are verbatim from the development. t.root is the root node of a tree, t.getSub p the subtree at position p, t.setSub p u the tree with that subtree replaced by u, t.pathSub p the honest sibling path, and t.get, t.set, t.path their leaf-level specialisations.

```
theorem fold_pathSub (t : Tree d) (p : Pos d h) :
    fold (t.getSub p).root (t.pathSub p) = t.root

theorem fold_setSub (t : Tree d) (p : Pos d h) (s : Tree h) :
    fold s.root (t.pathSub p) = (t.setSub p s).root

theorem root_inj (t₁ t₂ : Tree d) : t₁.root = t₂.root → t₁ = t₂

theorem fold_sound (t : Tree d) (m : Node H) (π : List (Bool × Node H)) :
    fold m π = t.root →
    ∃ (h : Nat) (p : Pos d h), m = (t.getSub p).root ∧ π = t.pathSub p
```

fold_sound is Theorem 1. Its proof is the only place the hash axioms are used: a non-empty path claims an internal node, so the tree is not a tip by tag separation, and injectivity of hNode equates both children's hashes and sums

with the claimed ones. Note that the conclusion equates *nodes*, so the sum the prover presented is the true sum of the subtree.

A Step is a path, a height, an old node and a new node; it is valid for roots r, r' when the old node folds to r and the new node to r', and well formed for depth d when the path has $d - h$ entries. Steps chains steps, each against the root the previous one produced.

```
theorem Step.sound (t : Tree d) (s : Step H) (r' : Node H)
    (hwf : s.WF d) (hv : s.valid t.root r') :
    ∃ p : Pos d s.height, s.old = (t.getSub p).root ∧
      ∀ (u : Tree s.height), u.root = s.new → r' = (t.setSub p u).root

theorem Steps.sum (h : Steps r ss r') :
    r'.sum + oldSum ss = r.sum + newSum ss

theorem Steps.sound (t : Tree d) (h : Steps t.root ss r')
    (hwf : ∀ s ∈ ss, s.WF d) (hg : ∀ s ∈ ss, s.Grounded) :
    ∃ t' : Tree d, t'.root = r'
```

Steps.sum is Theorem 2 and needs no hash assumption. Steps.sound is Theorem 4; s.Grounded says the new node is the root of some tree of the step's height, which leaf-level steps satisfy by construction (grounded_of_leaf) and which a JOIN graft must be given.

The transitions are structures bundling the steps with the output rule and an opaque authorisation. Their theorems:

```
theorem payable (t : Tree d) (p : Pos d 0) (l : Leaf) (hl : t.get p = some l) :
    (leafStep (t.path p) (some l) none).valid t.root (t.set p none).root ∧
    (t.set p none).root.sum + l.amount = t.root.sum ∧
    ∀ q, q ≠ p → (t.set p none).get q = t.get q

theorem ExitK.no_inflation (h : ExitK exact r blocks r' out0) :
    out0 + payoutSum blocks ≤ r.sum

theorem ExitK.exact_conservation (h : ExitK true r blocks r' out0) :
    out0 + payoutSum blocks = r.sum

theorem join_sound (t : Tree d) (hwf : path.length + h = d)
    (hj : Join auth h t.root path new r' out0) :
    ∃ p : Pos d h, t.getSub p = Tree.empty h ∧
      out0 = t.root.sum + new.sum ∧
      ∀ u : Tree h, u.root = new → r' = (t.setSub p u).root

theorem claim_sound (t : Tree d) (hwf₁ : srcPath.length = d)
    (hwf₂ : claimPath.length = d)
    (hc : Claim auth t.root srcPath claimPath src claim r' out0) :
    out0 = t.root.sum ∧
    ∃ (p₁ p₂ : Pos d 0), t.get p₁ = some src ∧
      (t.set p₁ (claimedSource src claim.amount)).get p₂ = none ∧
      r' = ((t.set p₁ (claimedSource src claim.amount))
              .set p₂ (some claim)).root
```

payable is Theorem 3. join_sound uses root_inj through eq_empty_of_root: since JOIN_h hard-codes E_h as the old node, the slot it fills is provably the empty tree, and the theorem gives the successor root for every tree whose root is the grafted node. claim_sound exposes the drained-source case: the claim position must be empty in the tree *after* the source step, which is why a source reduced to zero may host the claim.

A.3 How the Invariant Is Established and Maintained

The theorems are conditional on the current root being the root of a real tree. This invariant is established at creation: a tree is created by paying $\mathsf{TreeSPK}(r_0, m)$ with amount equal to the root sum of a tree the creator has built and, if it has members, whose members have verified their paths against r_0 before funding. It is maintained by Steps.sound for every grounded transition. The one transition that is not grounded by construction is a JOIN graft, and there the invariant is re-established by the joiners' off-chain check rather than by the covenant. Steps.sum holds without the invariant, so even a tree with an ungrounded node cannot inflate; it can only freeze the value under that node. This is the precise form of the informal claim that "the operator cannot steal": it cannot take, and it cannot make provable leaves unprovable, but it can graft value that was never provable to begin with, and only members who skipped their own check will have paid for it.

A.4 Binding to the Output

Binding.lean records the one cryptographic step outside the hash model. A structure BindingModel carries an abstract treeSpk : H → TapRoot → Spk with the hypothesis that it is injective in both arguments, and two one-line theorems conclude that a witness (r', m') passing the scriptPubKey check equals the committed (r, m), and that the successor output commits to the computed root under the same m. In the deployment, injectivity follows from the discrete logarithm assumption and collision resistance of H_{state} and the taproot tweak hash. Making the hypothesis explicit is the point: it is what a future script change must not break.

A.5 Encodings

Encoding.lean proves the byte-level facts the design leans on, over natural-number bytes so that every proof stays within linear arithmetic: fixed-width little-endian encodings are injective on their range; minimalLE (n + 2^(8k)) = leBytes k n ++ [1] for $n < 2^{8k}$, which with $k = 8$ is enc_{64} and justifies producing it with one addition; the 116-byte leaf and the 82-byte node preimage are injective; and the SEQ_FINAL facts of Lemma 1, computed by decide on a bounded-recursion model of `CScriptNum`.

A.6 What Is Not Proved

The tapscript bytecode is outside the model. That the scripts of §5 implement the relations above is established by execution: every script of every tier runs in the simulator against a reference tree with positive and negative cases, and the Go generators are compared byte for byte with the Python output. A verified interpreter for the tapscript subset (BIP-441 numerics, `OP_TX`, `OP_CHECKSIGFROMSTACK`) would close this gap and is a project of a different size. Signatures, lock times and the transaction structure beyond amounts are authorisation-side facts and

are deliberately opaque. Concurrency, mempool policy, reorganisations and the off-chain protocol are not theorems about hash trees and belong to a model checker; a design for a P [7] model is in the artefact.